
Lab Exercise: Auto-scaling Microservices with Spring Cloud and Kubernetes

Step 1: Set Up Spring Boot Projects

1. Create two Spring Boot applications:
 - user-service
 - order-service
2. Add dependencies for Eureka client, Spring Cloud LoadBalancer, and Actuator.

Step 2: Configure Service Discovery

1. Set up a Eureka server at localhost:8761.
2. Configure both services to register with Eureka using application.yml.
3. Start both services and verify they appear in the Eureka dashboard.

Step 3: Build Docker Images

1. Add Dockerfile to each service:

```
# Dockerfile for user-service
FROM openjdk:17-jdk-slim
COPY target/user-service.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Build images:

```
docker build -t yourname/user-service:latest -f Dockerfile .
docker build -t yourname/order-service:latest -f Dockerfile .
```

Step 4: Deploy to Kubernetes Cluster

1. Create Kubernetes deployment files:

```
user-service-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
```

```

        image: yourname/user-service:latest
        ports:
          - containerPort: 8080
        resources:
          limits:
            cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

```

Repeat similarly for order-service.

2. Apply deployments:

```

kubectl apply -f user-service-deployment.yaml
kubectl apply -f order-service-deployment.yaml

```

Step 5: Enable Horizontal Pod Autoscaler

1. Enable autoscaling for user-service:

```

kubectl autoscale deployment user-service --cpu-percent=50 --min=2 --max=10

```

This tells Kubernetes to maintain CPU utilization around 50% by scaling the number of pods between 2 and 10 .

2. Check status:

```

kubectl get hpa

```

Step 6: Simulate Load and Observe Scaling

1. Install Apache Benchmark:

```

sudo apt install apache2-utils

```

2. Generate load on user-service:

```

ab -n 100000 -c 100 http://<EXTERNAL_IP>/users/call-orders

```

3. Monitor pod count:

```

watch kubectl get pods

```

Observe the number of user-service pods increasing as load rises.

Complete Implementation

Final File Structure

```
lab-scaling-microservices/  
├── user-service/  
│   ├── src/  
│   │   └── main/  
│   │       ├── java/com/example/UserServiceApplication.java  
│   │       └── resources/application.yml  
│   ├── Dockerfile  
│   └── pom.xml  
├── order-service/  
│   ├── src/  
│   │   └── main/  
│   │       ├── java/com/example/OrderServiceApplication.java  
│   │       └── resources/application.yml  
│   ├── Dockerfile  
│   └── pom.xml  
├── user-service-deployment.yaml  
├── order-service-deployment.yaml  
└── README.md
```

README.md Summary

Scaling Microservices with Spring Cloud and Kubernetes

Overview

This lab demonstrates how to scale Spring Boot microservices using Spring Cloud and Kubernetes HPA.

Prerequisites

- Java 17
- Maven
- Docker
- Kubernetes cluster (Minikube or cloud provider)
- kubectl configured

Steps

1. Clone repository.
 2. Build services using Maven.
 3. Build Docker images.
 4. Deploy to Kubernetes.
 5. Enable autoscaling.
 6. Simulate load and observe scaling behavior.
-